

Laboratorium 10 – Obsługa baz danych

Języki skryptowe

Cele dydaktyczne

1. Zapoznanie się z obsługą relacyjnych baz w języku Python.
2. Zapoznanie się z mechanizmami odwzorowywania obiektowo-relacyjnego.

Wprowadzenie

Zapoznaj się ze zbiorem danych [Rozkład jazdy transportu publicznego \(GTFS\)](#). Zbiór danych zawiera paczkę w formacie GTFS. GTFS jest standardowym formatem opisu rozkładów jazdy komunikacji publicznej. Zbiór danych jest udostępniany jako paczka ZIP zawierająca kilka plików tekstowych. Pliki te mają rozszerzenie `.txt`, ale ich struktura odpowiada plikom CSV – dane są zapisane w kolumnach oddzielonych przecinkami.

Na potrzeby laboratorium wykorzystane zostaną wybrane pliki z paczki GTFS, opisujące linie, przystanki, kursy oraz godziny odjazdów z przystanków.

Paczka zawiera m.in. następujące istotne pliki:

- `stops.txt` – zawiera informacje o przystankach, m.in. identyfikator przystanku, kod przystanku, nazwę oraz współrzędne geograficzne,
- `routes.txt` – zawiera informacje o liniach komunikacyjnych, m.in. identyfikator linii, nazwę skróconą linii, opis trasy oraz typ środka transportu,
- `trips.txt` – zawiera informacje o konkretnych kursach realizowanych w ramach danej linii, m.in. identyfikator kursu, identyfikator linii, kierunek kursu oraz identyfikator kalendarza kursowania,
- `stop_times.txt` – zawiera informacje o kolejnych zatrzymaniach pojazdu w ramach danego kursu, m.in. identyfikator kursu, czas przyjazdu, czas odjazdu, identyfikator przystanku oraz kolejność przystanku na trasie,
- `calendar.txt` – zawiera informacje o dniach tygodnia, w których obowiązują wybrane kursy.

W uproszczeniu dane należy czytać następująco: linia komunikacyjna posiada wiele kursów, każdy kurs składa się z kolejnych zatrzymań na przystankach, a każde zatrzymanie ma przypisaną godzinę przyjazdu i odjazdu. Struktura najważniejszych plików obejmuje następujące kolumny:

Plik	Kolumny
stops.txt	• stop_id – identyfikator przystanku, • stop_code – kod przystanku, • stop_name – nazwa przystanku, • stop_lat – szerokość geograficzna, • stop_lon – długość geograficzna.
routes.txt	• route_id – identyfikator linii, • agency_id – identyfikator przewoźnika, • route_short_name – krótka nazwa linii, np. numer linii, • route_long_name – pełna nazwa linii, • route_desc – opis przebiegu trasy, • route_type – typ środka transportu.
trips.txt	• route_id – identyfikator linii, • service_id – identyfikator kalendarza kursowania, • trip_id – identyfikator kursu, • trip_headsign – kierunek kursu, • direction_id – identyfikator kierunku.
stop_times.txt	• trip_id – identyfikator kursu, • arrival_time – godzina przyjazdu na przystanek, • departure_time – godzina odjazdu z przystanku, • stop_id – identyfikator przystanku, • stop_sequence – kolejność przystanku w ramach kursu.
calendar.txt	• service_id – identyfikator kalendarza kursowania, • monday, tuesday, wednesday, thursday, friday, saturday, sunday – informacja, czy kurs obowiązuje w danym dniu tygodnia, • start_date – data początku obowiązywania, • end_date – data końca obowiązywania.

Przed przystąpieniem do wykonywania zadań zapoznaj się z sekcją **Wersja dla ambitnych** na końcu dokumentu.

Zadania

1. Na bazie struktury plików [GTFS](#) utwórz strukturę relacyjnej bazy danych w systemie zarządzania SQLite lub dowolnym innym RDBMS, która pozwoli na wstawienie, przechowywanie oraz odpytywanie danych o rozkładzie jazdy transportu publicznego. Baza danych powinna uwzględniać co najmniej tabele zawierające informacje o: przystankach, liniach komunikacyjnych, kursach, godzinach odjazdów i przyjazdów na przystankach.
 - a. Utwórz polecenia DDL/SQL ("CREATE TABLE...", "ALTER TABLE...") tworzące tę bazę danych w wybranym RDBMS. Skonstruuj klucze obce realizujące więzy integralności referencyjnej. Zastanów się nad odpowiednimi typami kolumn. W przypadku kolumn przechowujących godziny odjazdów i przyjazdów można wykorzystać typ tekstowy, np. TEXT, albo obliczyć pomocniczą wartość liczbową, np. liczbę sekund od początku doby.
 - b. Utwórz skrypt create_database.py, którego wywołanie spowoduje uruchomienie opracowanych poleceń DDL/SQL, a co za tym idzie, utworzenie bazy danych. Skrypt powinien przyjmować argument linii komend z nazwą bazy. Przykładowo, dla implementacji w SQLite wywołanie skryptu

```
python create_database.py my_timetable
```

powinno skutkować utworzeniem pliku my_timetable.sqlite3 zawierającego pustą strukturę.

2. Opracuj skrypt load_data.py, który jako argument przyjmie paczkę GTFS oraz plik z bazą danych, a następnie wstawi dane do bazy danych. Przykład wywołania:

```
python load_data.py OtwartyWroclaw_rozklad_jazdy_GTFS_28052026.zip my_timetable
```

Podczas importu należy zadbać o poprawną kolejność wstawiania danych, tak aby nie naruszać kluczy obcych. W przypadku większych plików, takich jak stop_times.txt, warto zastosować wstawianie danych w transakcji lub partiami.

3. Utwórz skrypt, który pozwoli użytkownikowi wybrać przystanek z listy, a następnie dla wskazanego przystanku skonstruuje zapytanie SQL typu "SELECT" obliczy i wyświetli:
 - a. liczbę różnych linii zatrzymujących się na danym przystanku,

- b. liczbę odjazdów z danego przystanku,
- c. najwcześniejszy i najpóźniejszy odjazd z danego przystanku,
- d. najczęstsze kierunki odjazdów z danego przystanku,
- e. wynik samodzielnie zaprojektowanego, nietrywialnego¹ zapytania.

Wersja dla ambitnych (na $\geq 75\%$ punktów)

Zadania 1–2 wykonaj dodatkowo z wykorzystaniem wybranej biblioteki do odwzorowywania obiektowo-relacyjnego, np.:

- SQLAlchemy ORM,
- Peewee,
- Tortoise ORM.

Wersja ORM powinna obejmować:

- definicję klas modeli odpowiadających tabelom bazy danych,
- odwzorowanie relacji między tabelami,
- utworzenie struktury bazy danych z poziomu kodu Pythona,
- import danych GTFS z użyciem modeli ORM,
- wykonanie co najmniej jednego zapytania agregującego z wykorzystaniem ORM.

Zadanie 3 wyposaż w graficzny lub webowy interfejsem użytkownika

¹ Nietrywialność oznacza tutaj np. połączenie danych z co najmniej dwóch tabel, grupowanie wyników, użycie funkcji agregujących, wykorzystanie podzapytania, itp.